Database Transactions

什麼是 Transaction？

在真實的系統裡，很少有操作是「一個步驟就完成」的。用戶轉帳，你需要把錢從 A 帳戶扣掉，再加到 B 帳戶，兩個步驟，缺一不可。用戶下訂單，你需要建立訂單記錄、扣庫存、產生付款請求，好幾個步驟，必須全部成功或全部失敗。

這就是 **Transaction（交易）** 要解決的問題：把一組相關的資料庫操作包成一個邏輯單元，保證它們作為一個整體被執行。

沒有 transaction，這中間的任何一步失敗，你的資料就會處於一個不一致的狀態，錢從 A 扣掉了，但沒有加到 B；訂單建立了，但庫存沒扣。這些都是生產環境裡真實存在的 bug，而且通常很難復現和修復。

Transaction 是資料庫系統的核心功能，讓你能把複雜的操作安全地封裝起來。

ACID：四個保證

ACID 是描述 transaction 必須提供哪些保證的縮寫。這四個屬性定義了「一個真正可靠的 transaction」長什麼樣。

Atomicity（原子性）

原子性保證：**一個 transaction 內的所有操作，要麼全部成功，要麼全部失敗。不存在中間狀態。**

轉帳的例子：如果從 A 扣款成功，但向 B 加款時失敗了，資料庫會 **rollback**（回滾），撤銷對 A 的扣款，讓資料回到 transaction 開始之前的狀態。對外來說，這個 transaction 就像從來沒有發生過。

資料庫怎麼實現這個？通常透過 **WAL（Write-Ahead Log）**：在真正寫入資料之前，先把「我打算做什麼」記錄到 log 裡。如果中途崩潰，重啟時根據 log 判斷該繼續完成還是回滾（rollback）。

Consistency（一致性）

一致性保證：**Transaction 執行完後，資料庫必須從一個合法狀態轉移到另一個合法狀態。** 任何你定義的資料完整性規則（constraints）都必須在 transaction 開始前和結束後成立。

buildmoat.org

舉例來說，你的資料庫設了一個 constraint：帳戶餘額不能為負數。一個想要把餘額從 100 扣成 -50 的 transaction 會被拒絕。

注意：Consistency 這個 C，和 CAP 定理裡的 C（所有節點看到相同資料）是不同的概念。這裡的 C 指的是資料的「商業邏輯層面的正確性」，而不是複本之間的同步。

Isolation（隔離性）

隔離性保證：**同時執行的多個 transactions，彼此不應該互相干擾。** 每個 transaction 看起來就像是在一個沒有其他 transaction 存在的環境下獨立執行。

這是四個屬性裡最複雜、也最有取捨空間的一個。完全的隔離性代價很高，所以實務上有不同等級的隔離，讓你在效能和安全性之間做選擇（後面會詳細說明）。

Durability（持久性）

持久性保證：**一旦 transaction commit 成功，資料就永久保存，即使系統立刻崩潰也不會遺失。**

這通常靠把資料寫入持久化儲存（磁碟）和維護 WAL 來實現。資料庫重啟後，可以從 log 重建已 commit 但尚未刷到磁碟的資料。

隔離等級（Isolation Levels）

隔離性是 ACID 裡最有彈性的部分。「完全隔離」代表所有 transactions 按順序一個一個執行，這很安全，但犧牲了大量的並行性（concurrency）。實務上，大多數資料庫支援四種隔離等級，讓你根據需求選擇：

隔離等級	Dirty Read	Non-repeatable Read	Phantom Read
Read Uncommitted	✅ 可能發生	✅ 可能發生	✅ 可能發生
Read Committed	❌ 防止	✅ 可能發生	✅ 可能發生
Repeatable Read	❌ 防止	❌ 防止	✅ 可能發生
Serializable	❌ 防止	❌ 防止	❌ 防止

Read Uncommitted

最低的隔離等級。Transaction 可以讀到其他尚未 commit 的 transaction 正在寫入的資料。

這是最危險的等級：如果另一個 transaction 之後 rollback，你讀到的就是一份從未真正存在過的資料。在生產環境幾乎不會使用這個等級。

Read Committed

buildnoat.org

Transaction 只能讀取已經 commit 的資料。這避免了 dirty read，是大多數資料庫（包括 PostgreSQL）的預設隔離等級。

但它還不夠完美，同一個 transaction 內，你兩次讀取同一筆資料，可能得到不同的結果（因為另一個 transaction 在這中間 commit 了更新）。

Repeatable Read

同一個 transaction 內，多次讀取同一筆資料，保證得到相同的結果。即使其他 transaction 在這期間更新了那筆資料，你也看不到變化。

MySQL InnoDB 的預設隔離等級是 Repeatable Read。

Serializable

最高的隔離等級。所有 transactions 的執行結果，等同於它們按照某個順序一個一個執行的結果。

最安全，但代價最高。通常只在金融交易等「絕對不能有並發異常」的場景才使用。庫存扣除的超賣問題本質上是 Lost Update，用 `SELECT FOR UPDATE` 或原子 UPDATE（`UPDATE SET qty = qty - 1 WHERE qty > 0`）就能解決，不需要提升到 Serializable。

三種並發異常

理解隔離等級，需要先理解它們在防止哪些問題：

Dirty Read（髒讀）

Transaction A 讀取了 Transaction B **尚未 commit** 的資料。如果 B 之後 rollback，A 就讀到了一份「從未存在」的資料。

```
T1: 把帳戶餘額從 1000 改成 500（尚未 commit）
T2: 讀取帳戶餘額，看到 500 ← Dirty Read
T1: 執行失敗，rollback，餘額回到 1000
T2: 錯誤地相信餘額是 500
```

Non-repeatable Read（不可重複讀）

同一個 transaction 內，兩次讀取同一筆資料，但兩次得到不同的結果，因為另一個 transaction 在這期間 commit 了對那筆資料的更新。

```
T1: 讀取帳戶餘額，得到 1000
T2: 把帳戶餘額從 1000 改成 500，commit
```

buildnoat.org

```
T1: 再次讀取帳戶餘額，得到 500 ← 和第一次不同
```

Phantom Read（幻讀）

同一個 transaction 內，兩次執行同樣的查詢（通常是範圍查詢），但第二次多出了新的資料行，因為另一個 transaction 在這期間 insert 了新資料。

```
T1: 查詢所有餘額 > 1000 的帳戶，找到 5 筆
T2: 新增一個餘額為 2000 的帳戶，commit
T1: 再次查詢，找到 6 筆  ← 多出了一筆「幽靈」資料
```

Non-repeatable Read 和 Phantom Read 的差別：前者是同一筆資料被改掉了，後者是新增或刪除了資料行，使得範圍查詢的結果集改變。

資料庫如何實現隔離：MVCC

資料庫不是靠「讓每個 transaction 等前一個完成」來實現隔離，那樣效能太差。現代資料庫（PostgreSQL、MySQL InnoDB）普遍使用 **MVCC（Multi-Version Concurrency Control，多版本並行控制）**。

核心思想很簡單：**對同一份資料保存多個版本。** 讀操作看的是資料在 transaction 開始時的「快照」，而不是最新版本，所以讀操作不需要等待寫操作完成，寫操作也不會被讀操作擋住。

```
帳戶餘額歷史版本：
  version 1：1000（T1 commit 時間點）
  version 2：500 （T2 commit 時間點）

T3 在 T2 commit 之前開始 → T3 看到 version 1（1000）
T4 在 T2 commit 之後開始 → T4 看到 version 2（500）
```

這讓讀寫可以並行，大幅提升了並行性。舊版本在沒有 transaction 需要它們之後，由資料庫的垃圾回收機制清理。

常見陷阱：Lost Update（更新遺失）

除了三種讀取異常，還有一個常見問題值得特別注意：**Lost Update**。

當兩個 transaction 同時讀取同一筆資料、各自計算新值、然後都寫回去時，後寫入的 transaction 會覆蓋前一個，讓前一個的更新完全消失。

buildnoat.org

```
帳戶餘額：1000

T1: 讀取餘額 1000，計算 1000 - 200 = 800，準備寫回
T2: 讀取餘額 1000，計算 1000 - 300 = 700，準備寫回
T1: 寫入 800，commit
T2: 寫入 700，commit  ← 覆蓋了 T1，T1 的扣款消失了！

正確結果應為 500，實際卻是 700
```

解法有兩種：

樂觀鎖（Optimistic Locking）：在更新時帶上版本號，如果版本號不符就拒絕更新。適合衝突不常發生的場景。

```
UPDATE accounts SET balance = 800, version = 2
WHERE id = 1 AND version = 1;  -- 如果 version 已被改掉，這行
影響 0 筆
```

悲觀鎖（Pessimistic Locking）：讀取時就把資料鎖住，阻止其他 transaction 讀取或修改，直到當前 transaction 完成。

```
SELECT balance FROM accounts WHERE id = 1 FOR UPDATE;  --
加排他鎖
-- 其他讀取 FOR UPDATE 的 transaction 會在這裡等待
UPDATE accounts SET balance = 800 WHERE id = 1;
COMMIT;
```

悲觀鎖較安全，但降低了並行性，也可能引發 **Deadlock（死鎖）**，兩個 transaction 互相等待對方釋放鎖。資料庫通常有 deadlock detection 機制，會 rollback 其中一個 transaction。

分散式 Transaction 的挑戰

以上討論的 transaction，都發生在單一資料庫內。當系統拆分成微服務、資料分散在多個資料庫中，情況就複雜得多。

舉個例子：電商系統裡，訂單服務用一個資料庫，庫存服務用另一個資料庫。建立訂單和扣庫存需要同時成功，但它們在不同的資料庫裡，沒有辦法用一個 BEGIN/COMMIT 把它們包在一起。

buildmoat.org

Two-Phase Commit（2PC，兩階段提交）

最直接的解法是 2PC：由一個協調者（coordinator）來確保所有參與節點都同意 commit。

第一階段（Prepare）：協調者問所有參與節點「你準備好 commit 了嗎？」每個節點把資料寫好但先不 commit，回傳 Yes 或 No。

第二階段（Commit/Abort）：如果所有節點都回 Yes，協調者發送 Commit 指令；只要有任何一個節點回 No，發送 Abort，所有節點 rollback。

2PC 的問題是：如果協調者在第二階段發出 Commit 前崩潰，部分節點可能永遠卡在「已 prepare 但未 commit」的狀態，需要人工干預。這讓 2PC 在高可用性要求的系統中難以使用。

Saga Pattern

現代微服務架構更常用的解法是 **Saga**：把一個分散式 transaction 拆成一系列小的、有補償機制的本地 transaction。

```
建立訂單流程：
1. 訂單服務：建立訂單記錄（本地 commit）
2. 庫存服務：扣減庫存（本地 commit）
3. 付款服務：扣款（本地 commit）

如果步驟 3 失敗：
  - 補償步驟 2：把庫存加回去
  - 補償步驟 1：把訂單標記為取消
```

Saga 放棄了強一致性，用「最終一致性」換取更好的可用性和容錯能力。這是絕大多數微服務系統面對分散式 transaction 的現實選擇。

系統設計面試中怎麼談 Transaction

Transaction 和 ACID 常在面試的設計討論中出現，特別是當你需要保證資料正確性時。

什麼時候主動提 Transaction

多步驟的原子操作：「建立訂單需要同時扣庫存和新增訂單記錄，我會把這兩個操作包在同一個 transaction 裡，確保它們要麼都成功要麼都失敗。」

需要防止並發更新錯誤：「多個用戶可能同時搶購最後一個庫存，我需要在讀取庫存時加鎖（SELECT FOR UPDATE），防止超賣。」

buildmoat.org

資料一致性要求高的場景：「這是一個金融系統，所有涉及帳戶餘額的操作都必須在 Serializable 隔離等級下執行。」

如何說明你的隔離等級選擇

別只說「我用 transaction」，要說清楚選哪個隔離等級，以及為什麼。

「對於讀取用戶個人資料這種操作，Read Committed 就夠了，不需要高隔離等級，也不想犧牲效能。」

「對於庫存扣除，超賣的本質是 Lost Update（兩個 transaction 都讀到舊庫存然後各自扣減），不是 Phantom Read。最簡單的解法是原子 UPDATE：`UPDATE SET qty = qty - 1 WHERE qty > 0`，一條 SQL 就搞定。如果扣減前需要做複雜判斷，再用 `SELECT FOR UPDATE` 鎖定該行，不需要提升到 Serializable。」

微服務場景：坦誠說明取捨

如果面試題目是微服務架構，面試官會期待你說明跨服務 transaction 的挑戰。

「我們的訂單服務和庫存服務用不同的資料庫。跨服務的強一致性很難保證，2PC 在高可用場景下不適合。我會改用 Saga 模式，每個服務做本地 transaction，失敗時執行補償操作，接受最終一致性，確保系統整體的可用性。」

主動說明死鎖（Deadlock） 風險

如果你用了悲觀鎖，面試官可能會問 deadlock 怎麼辦。提前說明比被問到更好：

「用 FOR UPDATE 鎖的時候，如果多個 transactions 以不同順序鎖定資源，可能發生 deadlock。解法是讓所有 transaction 都按照一致的順序獲取鎖，例如永遠先鎖 account_id 小的那個。資料庫本身的 deadlock detection 也會在偵測到死鎖後自動 rollback 其中一方並回傳錯誤，application 層需要處理重試。」

總結

Transaction 是保證資料庫操作可靠性的核心機制。ACID 四個屬性各自解決不同的問題：Atomicity 防止操作做到一半、Consistency 保證商業規則成立、Isolation 讓並發操作互不干擾、Durability 確保 commit 的資料不遺失。

四個屬性裡，**Isolation 是最有取捨空間的**。四種隔離等級從 Read Uncommitted 到 Serializable，保護的越多，代價越高。大多數系統預設 Read Committed，只在真正需要時才使用更高的隔離等級。

在分散式系統中，傳統的 ACID transaction 無法跨越服務邊界。2PC 提供強一致性但可用性差；Saga 放棄強一致性換取更好的容錯能力，是現代微服務的主流選擇。

buildmoat.org

在面試中，談到任何涉及多步驟操作、並發更新、或金融交易的設計時，都應該主動說明你的 transaction 策略，包括選什麼隔離等級、如何防止並發異常，以及跨服務時的替代方案。

自我測驗

Q1：ACID 四個字母分別代表什麼？各用一句話解釋。

> • **A**tomicity（原子性）：交易中的操作全部成功或全部回滾，不會只做一半。
> • **C**onsistency（一致性）：交易完成後，DB 從一個合法狀態轉到另一個合法狀態，所有 constraints 成立。
> • **I**solation（隔離性）：並發的交易之間不會互相干擾。
> • **D**urability（持久性）：已 commit 的資料在系統崩潰後仍然存在。

Q2：PostgreSQL 的預設隔離等級是什麼？MySQL InnoDB 呢？

> • PostgreSQL 預設：**Read Committed**
> • MySQL InnoDB 預設：**Repeatable Read**

Q3：配對題 — 將異常現象與定義配對：(1) Dirty Read (2) Non-repeatable Read (3) Phantom Read ／ (a) 同一交易中相同範圍查詢回傳不同數量的 rows (b) 讀到其他交易尚未 commit 的資料 (c) 同一交易中相同的 row 讀到不同的值

> • 1 → (b)：Dirty Read 是讀到尚未 commit 的資料。
> • 2 → (c)：Non-repeatable Read 是同一 row 的值被其他已 commit 的交易改變了。
> • 3 → (a)：Phantom Read 是新增或刪除了 rows，使範圍查詢結果集改變。

Q4：Lost Update 問題是什麼？Optimistic Locking 和 Pessimistic Locking 各自如何解決？

> **Lost Update**：兩個交易同時讀取同一筆資料、各自計算新值、都寫回去，後寫入的覆蓋了前一個的更新。
>
> - **Optimistic Locking**：使用版本號，更新時帶 `WHERE version = 原始版本`，版本不符就拒絕更新，由 application 重試。適合衝突不常發生的場景。
> - **Pessimistic Locking**：用 `SELECT ... FOR UPDATE` 鎖定 row，其他交易必須等待。較安全但降低並行性，可能造成 deadlock。

Q5：2PC 和 Saga Pattern 的主要差異是什麼？現代微服務更常用哪個？

buildneat.org

> ・**2PC（兩階段提交）**：由協調者確保所有節點都同意 commit，提供強一致性。問題是如果協調者崩潰，節點可能卡在「已 prepare 但未 commit」的狀態。
> ・**Saga Pattern**：把分散式交易拆成一系列本地交易加上補償操作，用最終一致性換取更好的可用性和容錯能力。
>
> 現代微服務更常用 **Saga Pattern**，因為 2PC 在高可用場景下太脆弱。

Q6：ACID 的 C 和 CAP 定理的 C 有什麼不同？

> ・**ACID 的 C（Consistency）**：指資料的「商業邏輯層面的正確性」，交易前後 constraints 必須成立（如餘額不能為負）。
> ・**CAP 的 C（Consistency）**：指「所有節點在同一時間看到相同的資料」，是分散式副本之間的同步問題。
>
> 兩者是完全不同的概念。

buildmoat.org